

Performance

A hands-on course in building agents by building their harness

Lesson 9

A run that finishes can still cost too much

- Lesson 8 made the agent keep going when something fails
- But ten minutes and ten dollars is a run you think twice about starting
- Over a long task two things add up: **the time you wait** and **the tokens you pay for**

Time and tokens come from the same place

Primitive	Where the time and tokens go
Context	what the request holds, resent on every pass
Model interface	which model a kind of request goes to
Output	waiting on commands to finish
Control flow	which model a turn of work goes to (routing)

Streaming isn't new: `call()` has streamed every response since Lesson 1.

The cost of a task is everything resent

- Every pass of the loop sends the **whole** request again: tools, instructions, every message
- Twenty steps means reading the same opening twenty times
- **Prompt caching:** the request is a prefix, read in a fixed order
- Mark where the remembering ends with `cache_control`; a call that starts with the same tokens only reads the new end
- A cache read costs a tenth of normal input; a write, a quarter more; it lasts five minutes since last use

A cache has three catches

- **The match has to be exact.** One changed character early and everything after it is new. That's why Lesson 4's prompt has the date, not the time
- **There's a minimum.** 512 to 4,096 tokens depending on the model; quark's system prompt is nine to eleven thousand
- **It belongs to one model.** Switch models and you start again

The system prompt has carried a mark since Lesson 4. New here: a second mark on the **last message**.

Send less, pick the model, wait together

- **Trim:** cap what one tool result may add, keeping the start and the end
- **Small jobs to the small model:** compaction always goes to a faster, cheaper one
- **Route by the work:** ask Jev how much work a request is, and pick the model from that
- **Run together:** several tool requests in one response run at the same time
- Results still go back in order, with their ids; questions first; one ESC stops all

The concept: route, cache, run together (abridged)

```
TIERS = {"lookup": "claude-haiku-4-5", "edit": "claude-sonnet-5-5", "work": "claude-opus-5-5"}
def route(input):
    try: size = jev.system_one({"input": input, {"size": SIZE}).choices["size"]
    except Exception: return TIERS["edit"]
    tier = size.choice if size.confidence >= 0.7 else "edit"
    print(f"[Jev: {size.choice}, {size.confidence:.2f}, so {TIERS[tier]}]")
    return TIERS[tier]

notes = [{"type": "text", "text": open("README.md").read(), "cache_control": {"type": "ephemeral"}}]
...
model = route(input)
print(call(model, input))
call(model, input)
...
with ThreadPoolExecutor() as pool: outputs = list(pool.map(run, checks))
```

The concept, run twice

```
[Jev: lookup, 0.99, so claude-haiku-4-5]  
[1.0s: 17 new, 8264 written to the cache, 0 read from it]  
[1.1s: 17 new, 0 written to the cache, 8264 read from it]  
[one at a time: 6.0s]  
lint ok, types ok, tests ok [at the same time: 2.0s]
```

```
[Jev: work, 0.64, so claude-sonnet-5-5]  
[4.3s: 41 new, 10695 written to the cache, 0 read from it]  
[4.6s: 41 new, 0 written to the cache, 10695 read from it]
```

(shortened: the answers are cut)

Model interface: a list of models (abridged)

```
MODELS, FAST = ["claude-sonnet-5-5", "claude-opus-5-5"], ["claude-haiku-4-5"]  
class Down(Exception): pass  
def call(each=lambda event: None, models=MODELS, **request):  
    for model in models:
```

Compaction always asks for the small model:

```
summary = call(models=FAST, max_tokens=2048, system=system(), messages=keep + [...])
```

Context: trim, and cache the end (abridged)

```
MAX_RESULT = 20_000
def trim(text):
    if len(text) <= MAX_RESULT: return text
    return text[:MAX_RESULT // 2] + f"\n[... {len(text) - MAX_RESULT} characters cut ...]\n" + ...

def cached(working_memory):
    last = working_memory[-1]
    blocks = [{"type": "text", "text": last["content"]} if isinstance(...) else list(last["content"])
    blocks[-1] = {"**blocks[-1], "cache_control": {"type": "ephemeral"}}
    return working_memory[:-1] + [{"role": last["role"], "content": blocks}]
```

```
response = call(unless_esc, models=models, ..., messages=cached(working_memory))
```

Output: one command becomes a function (abridged)

```
def execute(cmd):  
    if ESC.is_set(): return "[your doing never reached the world]"  
    for attempt in (1, 2):  
        ...  
        doing = subprocess.Popen(["docker", "exec", box, "timeout", ..., cmd], ...)  
        ...  
        if ESC.is_set(): done.stdout += "\n[your doing stopped before done]"  
        ...  
        if attempt == 2 or why != "transient" or not reads(cmd): break  
        print("[a failure that passes: trying once more]"); time.sleep(1)  
    if why == "partial": done.stdout += "\n(it may have partly run: check before repeating it)"  
    return trim(done.stdout) or f"(exit {done.returncode})"
```

Decide each request first, in order (abridged)

```
refused, pending, lent = {}, {}, set()
for block in output:
    if block.type == "tool_use":
        ...
        if not cmd or (response.stop_reason == "max_tokens" and block.is output[-1]):
            refused[block.id] = "[your doing was cut off before it was fully formed – ...]"
        elif (no := guard(cmd)):
            trace(event="refused", cmd=cmd, why=no)
            refused[block.id] = no
        else:
            pending[block.id] = cmd
            if lend(cmd): lent.add(block.id)
```

Then run everything allowed, at once (abridged)

```
with listening(), ThreadPoolExecutor() as pool:
    outputs = dict(zip([i for i in pending if i not in lent], pool.map(execute, [...])))
    for i in lent:
        bridge(True); outputs[i] = execute(pending[i]); bridge(False)
```

- One ESC reaches every running command: `kill -9 -1` in the box
- A command lent the network runs on its own, with the way out open just for it
- Results go back in the order asked, each with its id
- New prompt line: *they run at the same time*

Control flow: route each turn (abridged)

```
TIERS = {"lookup": FAST + MODELS, "edit": MODELS, "work": MODELS[::-1]}  
def route(input):  
    size = ask({"input": input}, SIZE) if isinstance(input, str) and input else None  
    tier = size["choice"] if size and size["confidence"] >= 0.7 else "edit"  
    trace(event="routed", to=TIERS[tier][0], size=..., confidence=...)  
    return TIERS[tier]
```

```
steps, spent, models = 0, 0, route(input)
```

- `SIZE` is the concept's question; lookup → Haiku, edit → Sonnet, work → Opus first
- Unsure, no key, no answer: `edit`, Lesson 8's models
- Once per person turn: switching mid-turn would throw the cache away

Three seconds of waiting, not nine

```
$ sleep 3; echo lint ok
allow `sleep 3; echo lint ok`? (Jev: read, 0.88) [y/N] $ sleep 3; echo types ok
allow `sleep 3; echo types ok`? (Jev: read, 0.84) [y/N] $ sleep 3; echo tests ok
allow `sleep 3; echo tests ok`? (Jev: read, 0.75) [y/N] lint ok

types ok

tests ok

Lint, types and tests all passed.
[wall: 10.6s]
```

Lesson 8's `quark.py` ran them one after another: `[wall: 16.5s]` .

The trace shows it, and the cache too (shortened)

Lesson 8:

```
{"event":"model","seconds":1.74,"input_tokens":145,"cache_read":0,"cache_write":10391}  
{"event":"model","seconds":0.93,"input_tokens":407,"cache_read":10391,"cache_write":0}
```

This lesson:

```
{"event":"routed","to":"claude-sonnet-5-5","size":"work","confidence":0.51}  
{"event":"model","seconds":1.8,"input_tokens":4,"cache_read":0,"cache_write":11342}  
{"event":"model","seconds":1.47,"input_tokens":2,"cache_read":11342,"cache_write":264}
```

Pay full price for what's new, and keep it short (shortened)

A three-step task, one command per step:

```
{"event":"routed","to":"claude-opus-5-5","size":"work","confidence":0.83}  
{"event":"model","seconds":2.04,"input_tokens":4,"cache_read":0,"cache_write":11330}  
{"event":"model","seconds":1.84,"input_tokens":2,"cache_read":11330,"cache_write":158}  
{"event":"model","seconds":5.08,"input_tokens":2,"cache_read":11488,"cache_write":125}
```

`cat docs/the-model.md`, over 25,000 characters:

```
{"event":"tool","cmd":"cat docs/the-model.md","seconds":0.11,"exit":0,"chars":25435,"failed":0.03}  
{"event":"model","seconds":0.83,"input_tokens":6,"cache_read":9051,"cache_write":4936}
```

One ESC stops both commands (shortened)

```
$ for i in 1 2 3 4 5 6 7 8 9; do echo a$i; sleep 1; done
$ for i in 1 2 3 4 5 6 7 8 9; do echo b$i; sleep 1; done
a1
...
a4

[your doing stopped before done]
b1
...
b4

[your doing stopped before done]
```

Then the model: *"You interrupted me, so both commands stopped early."*

Routing, turn by turn (shortened)

A chat: *how many lines is slow.py?*, then *why is it slow*, and *what would you change?*

```
{"event": "routed", "to": "claude-haiku-4-5", "size": "lookup", "confidence": 1.0}
{"event": "model", "seconds": 0.93, "input_tokens": 3, "cache_read": 0, "cache_write": 9038}
{"event": "model", "seconds": 0.45, "input_tokens": 6, "cache_read": 9038, "cache_write": 84}
{"event": "routed", "to": "claude-opus-5-5", "size": "work", "confidence": 0.9}
{"event": "model", "seconds": 1.72, "input_tokens": 4, "cache_read": 0, "cache_write": 11427}
```

The switch to Opus started its cache over: `cache_read` 0.

Other things we could do

- **What gets cached:** up to four marks; a one-hour lifetime for agents that wait on people
- **What goes in, and comes back:** clear old results; fetch on demand; "be brief"; less thinking
- **How many at once:** cap parallel tools; the batch interface at about half the price
- **Measure what a person feels:** time to first output, and the share read from the cache
- **As a product:** the API's own settings; [LiteLLM](#) and [OpenRouter](#) choose models per request

The rule

Don't resend what hasn't changed, don't send what isn't needed, and don't wait for things one at a time that could be waiting together.

Performance changes how, never what

- **Control flow:** the same loop and stops; routing only picks a turn's models
- **Input:** the person's request is read the way it always was
- **Context** still decides what the request holds, only marked for reuse
- **Model interface** still sends a request and gets a response; asking Jev changes nothing on its own
- **Output** still runs what the response asks for, only together, and stopped together

Faster and cheaper, but as good?

- A trimmed result might have cut the line that mattered
- A summary from the small model might have lost a fact
- The router might send a request to a model too weak for it: Haiku described the wrong heading
- Parallel commands can quietly depend on each other
- The trace shows a faster, cheaper run, and not whether it did the job

Next: Evaluation

Lesson 10